

PRISM: Confidential Smart Contract Auditing via Pseudonymized Graph Screening and Local Dynamic Verification

Anonymous Author(s)

Anonymous Institution

Abstract. Large language models (LLMs) are increasingly used to audit Ethereum Virtual Machine (EVM) smart contracts, but hosted models require auditors to send unreleased source code, and the business logic it encodes, to a third party. We present PRISM, a confidential audit pipeline whose trust boundary is the audit host. PRISM strips secrets and reversibly pseudonymizes identifiers, turns Slither’s static single assignment (SSA) intermediate representation into typed function-level code property graphs (CPGs), screens and ranks functions with a two-stage graph attention model, audits the highest-ranked functions with a locally served open-weight code LLM, and exercises them with Foundry invariant harnesses on a local EVM, restoring identifiers only in the final report. Experiments on 12,991 public function graphs covering five Smart Contract Weakness Classification (SWC) classes show that the ranker reaches $75.95 \pm 0.19\%$ macro-F1 on a random split and $76.31 \pm 0.06\%$ on a lineage-disjoint split over five seeds, at 0.21 ms per function. Graph convolutional and graph isomorphism baselines reach the same level, share at least 97% of its errors and differ in none of 20 paired tests, which places the accuracy ceiling in contract-derived labels rather than in model capacity. On a 57-contract benchmark, pseudonymization keeps every contract compilable and every Slither detector type, yet an adversary who knows plaintext identifier frequencies re-links 89.5% of identifiers, so masking complements local inference but cannot replace it. Taxonomy- and severity-aware filtering lowers the share of safe contracts with any alarm from 27/27 to 6/27 while 27/30 vulnerable contracts remain flagged.

Keywords: Smart contract security · Confidential auditing · Graph neural networks · Local large language models · Invariant fuzzing · Pseudonymization.

1 Introduction

Smart contracts on the Ethereum Virtual Machine (EVM) hold large amounts of value in decentralized finance, and deployed bytecode cannot be patched in place [1, 16]. Flaws must therefore be found before deployment, usually by auditing source code that its owner has not yet released and considers proprietary [1].

Large language models (LLMs) are now the most active direction in automated auditing. GPTScan [24] and PropertyGPT [14] pair hosted GPT models

with program analysis or formal verification, iAudit [17] combines fine-tuned detectors with LLM agents, LLM-SmartAudit [28] organizes multi-agent audits, and PromFuzz [12] and LLAMA [7] use LLMs to derive invariants or seeds for fuzzing. Graph learning has advanced in parallel, from graph neural network (GNN) detectors over contract graphs [15, 34] to attention over local and global context [22] and heterogeneous graph transformers with LLM-derived features [19]. Classical tools explain why learning is attractive: 97% of 47,587 contracts were flagged by at least one static analyzer [4], and fuzzers need guidance to pass narrow guards [3, 23].

Most LLM auditors, however, call hosted models [14, 24, 28], so the auditor transmits unreleased code, identifier names and business logic to a model provider. Provider-side confidential inference remains a research prototype [11], and open-weight models fine-tuned for vulnerability detection [18] stop at a label without executable evidence. An auditor who must keep code in house therefore faces two open questions: which parts of an LLM-assisted audit can run on commodity local hardware while still producing executable evidence, and whether cheap pseudonymization of identifiers would make hosted models acceptable after all. Learned models already recover variable names from stripped binaries [30, 31], which suggests that renaming is fragile, but its residual exposure has not been measured for smart contract audits.

PRISM builds on the observation that the costly or risky steps of an LLM-assisted audit need different guarantees. Deciding *where* to look is a ranking problem that a compact graph model solves in well under a millisecond per function. Proposing *what* may be wrong benefits from a code LLM but does not need a frontier model when its output is treated as a hypothesis. Deciding *what is real* requires execution. PRISM therefore places a two-stage graph attention ranker in front of a locally served open-weight code LLM and a Foundry execution back end, and keeps every model and every artifact on the audit host; identifiers are pseudonymized before any learned stage and restored only in the final report. Our contributions are:

- We design PRISM, a confidential audit pipeline with an explicit two-adversary threat model in which secret filtering, reversible pseudonymization, graph screening, hotspot-conditioned local LLM auditing, Foundry execution and template repair all run on the audit host (Sect. 3). Among the systems we survey in Sect. 2, it is the only one that runs its learned stages locally, protects identifiers before model input and confirms findings by execution.
- We introduce a function-level code property graph (CPG) over Slither’s static single assignment (SSA) intermediate representation with typed control- and data-flow edges, and a two-stage graph attention ranker. We show over five seeds that it ranks functions at $76.31 \pm 0.06\%$ macro-F1 on unseen contract lineages in 0.21 ms per function, and that graph learners of different design converge to the same ceiling and share at least 97% of their errors, which identifies contract-derived labels as the limiting factor (Sect. 5.1).
- We measure what reversible pseudonymization preserves and what it leaks. Masked code keeps compilation (57/57) and all 229 contract-level Slither

detector types, whereas a frequency-aware adversary re-links 89.5% of identifiers, which quantifies why masking must be paired with local inference (Sect. 5.3).

- We evaluate the local execution and filtering stages on a 57-contract benchmark. All 300 seeded Foundry runs complete reproducibly, taxonomy- and severity-aware filtering reduces flagged safe contracts from 27/27 to 6/27 while 27/30 vulnerable contracts stay flagged, and five template patches pass five validation checks (Sects. 5.2, 5.4 and 5.5).

The remainder of this paper is organized as follows. Sect. 2 reviews related work, Sects. 3 and 4 present the design and setup, Sect. 5 answers five research questions, and Sects. 6 and 7 discuss and conclude.

2 Related Work

Static analysis and fuzzing. Oyente [16] applies symbolic execution, Securify [25] checks Datalog patterns, and Slither [5] runs rule-based detectors on SlithIR, an SSA intermediate representation (IR); empirical studies report low precision and uneven recall for such tools [4, 8]. ContractFuzzer [10], Echidna [9], sFuzz [21], Harvey [29], Smartian [3] and ItyFuzz [23] contribute call-level oracles, property-based, adaptive, greybox, data-flow-guided and snapshot-based fuzzing.

Graph learning. Zhuang et al. [34] and Liu et al. [15] learn over contract graphs with normalized nodes and expert patterns, DA-GNN [32] applies dual attention, ConSense [22] combines local and global attention, and MANDO-LLM [19] feeds LLM code features into heterogeneous graph transformers. PRISM’s ranker builds on graph attention networks (GAT) and their GATv2 variant [2, 26] and operates on a homogeneous SSA-level function graph with typed edge attributes. Its role is to decide which functions later stages examine, not to issue the final verdict.

LLMs for auditing, fuzzing and repair. GPTScan [24] pairs GPT with static confirmation, PropertyGPT [14] and SmartInv [27] generate properties or invariants for verification, and LLM-SmartAudit [28] and iAudit [17] organize LLM agents around the audit. PromFuzz [12] and LLAMA [7] use LLMs to guide fuzzing. EVuLLM [18] fine-tunes small open-weight models with QLoRA. On repair, sGuard [20] and Elysium [6] patch source or bytecode with templates.

Confidential analysis. Provider-side protection of prompts with confidential computing is an active topic [11]. On the attack side, ReSym [30] and GenNm [31] recover variable names and types from stripped binaries with fine-tuned LLMs, showing that removing or renaming identifiers does not remove the information that structure carries. PRISM takes the conservative position: inference stays local, and pseudonymization protects only artifacts that leave the host.

Table 1 summarizes the capabilities. PRISM is not the strongest detector or fuzzer in isolation; its distinguishing property is the combination of local learned inference, identifier protection and execution-based confirmation.

Table 1. Capability comparison with representative systems, ordered by year and name. GNN: learned graph detector; LLM: uses an LLM; Local: open-weight model run on premises; Privacy: protects identifiers or source before model input (for PRISM, by local inference; pseudonymization protects only artifacts that leave the host, Sect. 5.3); Execution: confirms findings by execution (F: formal proof instead); Repair: synthesizes patches; n/a: no model is used; NR: not reported.

System	Year	GNN	LLM	Local	Privacy	Execution	Repair
Slither [5]	2019	✗	✗	n/a	n/a	✗	✗
sGuard [20]	2021	✗	✗	n/a	n/a	✗	✓
Smartian [3]	2021	✗	✗	n/a	n/a	✓	✗
Elysium [6]	2022	✗	✗	n/a	n/a	✗	✓
ItyFuzz [23]	2023	✗	✗	n/a	n/a	✓	✗
GPTScan [24]	2024	✗	✓	✗	✗	✗	✗
EVuLLM [18]	2025	✗	✓	✓	✗	✗	✗
PromFuzz [12]	2025	✗	✓	NR	✗	✓	✗
PropertyGPT [14]	2025	✗	✓	✗	✗	F	✗
LLAMA [7]	2026	✗	✓	NR	✗	✓	✗
MANDO-LLM [19]	2026	✓	✓	NR	✗	✗	✗
This work (PRISM)	2026	✓	✓	✓	✓	✓	✓

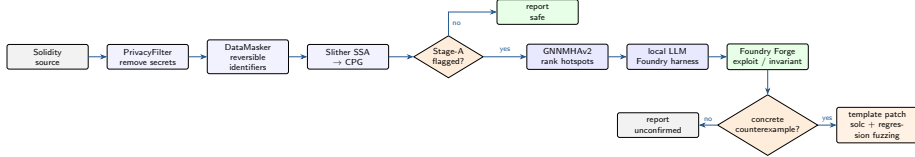


Fig. 1. PRISM architecture. All eight stages run inside the audit host. Ranked functions are audited by a local open-weight LLM, whose findings are hypotheses, and exercised by Foundry invariant harnesses, which confirm a finding only through a failing assertion. $\mathcal{T}$: pseudonymization map; Φ : ranking score; τ : forwarding threshold.

3 PRISM Design

Figure 1 shows the pipeline; no network access is required once the model weights are installed.

3.1 Threat Model and Pseudonymization

Assets and trust boundary. The asset is unreleased contract source code together with the business logic it reveals: identifier names, constants, addresses and endpoints. The trust boundary is the audit host operated by the contract owner or its auditor.

Adversaries. We consider (A1) third-party model providers and network observers, to which a cloud-LLM workflow would expose code, and (A2) honest-but-curious parties that later see PRISM artifacts outside the host, such as

prompt logs, generated harnesses, bug reports shared with external reviewers, or a shared inference server inside the organization. PRISM removes A1 by construction, since no inference is remote, and DataMasker reduces what A2 learns from artifacts. A fully compromised audit host is out of scope, because such an adversary can read the plaintext source directly.

Mechanism. *PrivacyFilter* first removes hard-coded private keys and application programming interface (API) keys, remote procedure call (RPC) endpoints and 20-byte addresses. *DataMasker* then renames user-defined identifiers of contracts, types, functions and variables, and leaves Solidity keywords, EVM built-ins such as `msg.sender` and `block.timestamp`, standard interfaces such as `IERC20`, numeric literals and comments untouched. The i -th distinct identifier x_i is mapped to

$$\mathcal{T}(x_i) = \text{prism_}\sigma(x_i)_ \langle i \rangle, \quad (1)$$

where $\langle i \rangle$ is a four-digit counter and $\sigma(x) \in \{\text{Type}, \text{b}, \text{v}\}$ keeps a coarse lexical role: **Type** for capitalized names, **b** for boolean-style names beginning with *is*, *has* or *can*, and **v** otherwise. $\mathcal{T}$ is a bijection; its inverse $\mathcal{T}^{-1}$ never leaves the host and is applied only to the final report. Sect. 5.3 measures what $\mathcal{T}$ preserves and what it still reveals to A2.

3.2 Function-Level Code Property Graph

From the pseudonymized source S_{mask} , PRISM extracts Slither’s SSA IR [5] and builds one directed graph per function, $\mathcal{G} = (\mathcal{V}, \mathcal{E}, X, E_{attr})$, whose nodes are SSA instructions.

- **Node features** $X \in \mathbb{R}^{N \times 32}$: a 13-dimensional one-hot instruction type, 10 security indicators (low-level calls, storage reads and writes, guards, transaction-context reads, etc.), and 9 structural attributes (in- and out-degree, loop membership, relative depth, etc.).
- **Edge attributes** $E_{attr} \in \{0, 1\}^{|\mathcal{E}| \times 4}$: a one-hot edge type among **sequential**, **branch**, **loop_back** (control flow) and **data_dep** (def-use), so that control and data flow share one edge set.

3.3 Two-Stage Graph Attention Ranker

Both stages use the same encoder, GNNMHA_{v2}, which projects node features to $d = 256$ dimensions and stacks $L = 4$ edge-aware graph attention layers with $K = 8$ heads [2, 26]. With $\tilde{\mathbf{e}}_{ij} = W_e \mathbf{e}_{ij}$ the projected edge attribute and $\mathcal{N}^+(i) = \mathcal{N}(i) \cup \{i\}$, head k of layer l computes

$$\alpha_{ij}^{k,l} = \text{softmax}_{j \in \mathcal{N}^+(i)} \left(\text{LeakyReLU}(\mathbf{a}_s^{k\top} W^{k,l} h_i^l + \mathbf{a}_t^{k\top} W^{k,l} h_j^l + \mathbf{a}_e^{k\top} \tilde{\mathbf{e}}_{ij}) \right), \quad (2)$$

$$h_i^{l+1} = \text{ELU} \left(\text{BN} \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}^+(i)} \alpha_{ij}^{k,l} W^{k,l} h_j^l \right) \right) + h_i^l, \quad (3)$$

where BN is batch normalization and ELU the exponential linear unit. Residual connections and a jumping-knowledge (JK) readout, $z_i = [h_i^1 \| h_i^2 \| h_i^3 \| h_i^4] \in$

$\mathbb{R}^{1024}$, keep both local and multi-hop patterns. The graph embedding $g \in \mathbb{R}^{2048}$ concatenates attention pooling and mean pooling,

$$g = \left[\sum_i \beta_i z_i \parallel \frac{1}{N} \sum_i z_i \right], \quad \beta_i = \text{softmax}_i (\mathbf{w}^\top \text{ReLU}(W_p z_i)), \quad (4)$$

and a two-hidden-layer perceptron maps g to class logits. One layer costs $O(K(|\mathcal{V}|d^2 + |\mathcal{E}|d))$, which is linear in graph size for fixed d .

Stage A is a binary model (vulnerable or safe) trained with class-weighted, label-smoothed cross-entropy (CE). Stage B is a five-class model trained with class-weighted focal loss on label-smoothed CE, with smoothing $\epsilon = 0.05$ and $\gamma = 2$:

$$\mathcal{L} = \frac{1}{n} \sum_{m=1}^n \alpha_{y_m} (1 - p_m)^\gamma \text{CE}_\epsilon(g_m, y_m), \quad (5)$$

where p_m is the predicted probability of the true class and α_y the class weight. At inference, Stage A drops functions it predicts safe, Stage B assigns a class to the remaining ones, and PRISM ranks them by the Stage-B confidence $\Phi(\mathcal{G}) = \max_c p_B(c \mid \mathcal{G})$, forwarding functions with $\Phi \geq \tau$ ($\tau = 0.3$) in decreasing order of Φ .

3.4 Local LLM Audit, EVM Execution and Repair

Local LLM audit. The masked source and the ranked functions, each with its predicted class and Φ , are passed to an open-weight code LLM, DeepSeek-Coder-V2-Lite-Instruct, a 16B-parameter mixture-of-experts model with 2.4B active parameters [33]. It is served on the host by Ollama¹ with temperature 0.1 and at most 4,096 generated tokens. The prompt asks for a structured finding per suspected weakness: weakness type, severity, affected function, attack path, exploit scenario and fix. PRISM treats these findings as hypotheses.

Execution. For each ranked contract, PRISM generates a Foundry test contract that inherits `forge-std/Test.sol` and instantiates, for every ranked function, the invariant template of its predicted class, for example balance conservation after withdrawal for reentrancy and rejection of privileged calls from non-owners for access control. Targeted seeds for the ranked functions are mixed with mutated seeds from a seed pool, and a coverage feedback loop repeats the campaign until coverage stops improving or reaches a target. A finding is *confirmed* only if an assertion fails with a concrete counterexample trace.

Repair and report. A template-based repair module proposes a patch for each finding, which must pass five checks (Sect. 5.5). LLM hypotheses and execution results are merged and deduplicated by function and weakness type, and identifiers are restored through $\mathcal{T}^{-1}$ only in the final report.

¹ <https://ollama.com>; the quantized model occupies 8.3 GiB on disk.

4 Experimental Setup

Research questions (RQs). **RQ1:** How accurately does the ranker classify vulnerable functions, on seen and unseen contract lineages, and how does it compare with other graph learners? **RQ2:** Does the local EVM back end execute harnesses reliably and reproducibly? **RQ3:** Does pseudonymization preserve analysis utility, and what does it still reveal? **RQ4:** How far do taxonomy- and severity-aware filters reduce alarms on safe contracts without losing vulnerable ones? **RQ5:** Do template patches block exploits while preserving interfaces?

Dataset. We extract 12,991 function-level CPGs with at least three nodes from three public sources: SmartBugs Wild [4] (1,179 graphs), the SolidiFI benchmark [8] (8,105 graphs) and LISA-Bench [13] (3,707 graphs), a collection of professional audit findings. The graphs cover five classes of the Smart Contract Weakness Classification (SWC) registry: Reentrancy (SWC-107, 2,475), Integer Overflow (SWC-101, 2,392), Access Control (SWC-105, 2,026), Unchecked Return Value (SWC-104, 2,983) and front-running by transaction-order dependence (SWC-114, 3,115). Graphs of the remaining source categories (denial of service, timestamp dependence, delegatecall) are excluded from the five-class task. Labels follow a contract-derived weak-supervision policy: SmartBugs and SolidiFI functions inherit the label of their contract, and LISA-Bench functions inherit the label obtained by keyword mapping of the finding text. We audited this choice on SolidiFI: of 9,369 injected bug spans in its 350 BugLog contracts, 4,928 map uniquely to a function body, and a strictly span-mapped subset would keep only 2,580 graphs with two Reentrancy and two front-running examples, too few for a five-class task. The resulting label noise is discussed in Sect. 6. Stage A uses 3,669 binary graphs (3,132 vulnerable, 537 safe).

Splits. We use two frozen protocols. (i) *Random split*: stratified 70/15/15 (9,093/1,949/1,949 graphs). (ii) *Lineage-disjoint split*: contracts are partitioned by origin so that SolidiFI `buggy_N` variants, functions from the same LISA-Bench finding, and functions from the same imported source contract never cross partitions, giving 8,349/2,496/2,146 graphs with zero lineage overlap.

Baselines. To isolate the effect of the encoder, we train graph convolutional (GCN) and graph isomorphism (GIN) networks as well as GAT and GATv2 on the same graphs, node features, splits and training budget; GAT and GATv2 do not use edge attributes. As a rule-based reference, Slither 0.11.5 is mapped to the five classes through its detector names (e.g., `reentrancy-eth` to Reentrancy, `unchecked-lowlevel` to Unchecked Return), and a function that triggers no mapped detector is assigned the majority training class.

Implementation. Models are implemented in PyTorch Geometric and trained with AdamW (learning rate 10^{-3} , weight decay 10^{-4}), batch size 64 and OneCycleLR for up to 35 epochs with early stopping on validation macro-F1 (patience

Table 2. Comparison with graph baselines on both splits. Five-seed rows report mean $\pm$ standard deviation; GAT and GATv2 are single runs (seed 42). Slither is mapped to the five classes with majority-class fallback. Best mean per column in bold.

Method	Random split ($n = 1,949$)		Lineage-disjoint split ($n = 2,146$)	
	Accuracy	Macro-F1	Accuracy	Macro-F1
Slither 0.11.5 (mapped)	0.2299	0.0748	0.2330	0.0756
GAT (seed 42)	0.7399	0.7562	0.7437	0.7611
GATv2 (seed 42)	0.7409	0.7571	0.7437	0.7621
GCN (5 seeds)	0.7434 ± 0.0016	0.7604 ± 0.0022	0.7445 ± 0.0008	0.7630 ± 0.0009
GIN (5 seeds)	0.7442 ± 0.0005	0.7609 ± 0.0005	0.7432 ± 0.0011	0.7617 ± 0.0011
GNNMHA2 (PRISM, 5 seeds)	0.7426 ± 0.0014	0.7595 ± 0.0019	0.7445 ± 0.0006	0.7631 ± 0.0006

15). GNNMHA2, GCN and GIN are trained with five seeds (42 to 46); GAT and GATv2 with seed 42. Experiments run on an Apple M4 CPU (10 cores, 16 GB RAM). One 35-epoch training run of GNNMHA2 takes 348.3 ± 37.2 s, compared with 56.5 ± 6.7 s for GCN and 67.4 ± 1.3 s for GIN, and inference takes 0.21 ms per function graph. Execution uses Foundry Forge 1.5.1 and `solc` 0.8.30.

5 Results

5.1 RQ1: Function-Level Ranking Accuracy

Table 2 compares the ranker with graph baselines. GNNMHA2 reaches 0.7595 ± 0.0019 macro-F1 on the random split and 0.7631 ± 0.0006 on the lineage-disjoint split, with macro-precision 0.887 ± 0.005 and macro-recall 0.725 ± 0.001 on the random split. The lineage-disjoint score is not lower than the random-split score, so performance does not depend on shared mutant origins. The mapped Slither reference scores 0.0748 because its detectors fire on few function slices under our label mapping; its accuracy therefore equals the majority-class rate.

The more informative result is that all graph learners converge to the same level. On both splits, GCN, GIN, GAT, GATv2 and GNNMHA2 lie within 0.5 macro-F1 points of each other. In 20 paired exact McNemar tests between GNNMHA2 and GCN or GIN (two splits, five seeds), no difference is significant (all $p > 0.16$), the paired models disagree on at most 22 test functions per run, and between 97.4% and 99.6% of GNNMHA2’s errors are shared with the baseline. When models of different inductive bias make the same mistakes on the same functions, the errors are a property of the data, not of the encoder.

The per-class view in Fig. 2 shows where the shared errors go. Precision is 0.95 to 1.00 for the four specific classes, but their recall is 0.56 to 0.73, and 490 of the 505 errors are predictions of *unchecked return* (precision 0.47, recall 0.99), whose SSA signature, an external call followed by an unused result, is the most generic. The model therefore rarely confuses two specific weaknesses. It assigns functions that carry no class-specific evidence to the most generic class, which is what contract-derived labels produce when a contract’s label is inherited by functions that do not contain the flaw. Consistently, top-2 accuracy (five-seed

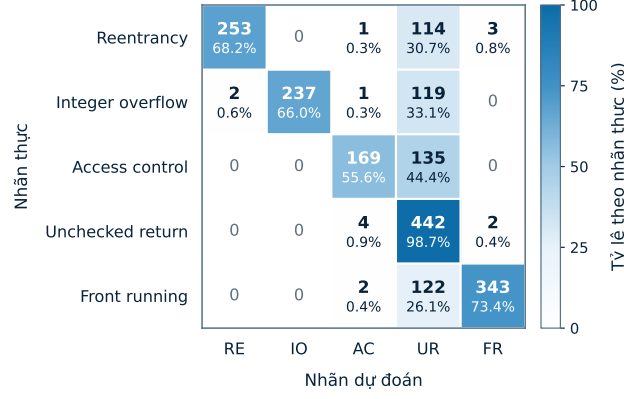


Fig. 2. Confusion matrix of a single GNNMHA v2 training run on a stratified 70/15/15 random split (seed 42) that precedes the frozen split manifest ($n = 1,949$; accuracy 0.7409, macro-F1 0.7577; five-seed results are in Table 2). Cells show counts and the share of the true class. Columns follow the row order: Reentrancy (RE, SWC-107), Integer overflow (IO, SWC-101), Access control (AC, SWC-105), Unchecked return (UR, SWC-104), Front-running (FR, SWC-114).

mean) is 0.8086, so the second-ranked class recovers only 25.6% of the errors: most errors are not near misses.

The ablation in Table 3 supports the same reading. Removing edge attributes, jumping knowledge or attention pooling, replacing focal loss, and changing depth or head count all move macro-F1 by at most 0.16 points, within one to two standard deviations of seed variation. Under contract-derived labels, encoder design is not the bottleneck; label granularity is. For PRISM this has a practical consequence. The ranker’s job is to order functions for later stages, any of these encoders does so at the same quality, and at 0.21 ms per function the choice costs nothing next to execution. Better labels, not a larger encoder, are the lever for better ranking.

5.2 RQ2: Execution Back-End Reliability

We executed expert-written reference harnesses for five case-study contracts under 30 fuzzing seeds (101 to 130) in two modes, a directed exploit test and a 10,000-call fuzzing campaign, for $5 \times 30 \times 2 = 300$ runs (Table 4). Four targets cover taxonomy classes; *WeakLottery* (SWC-120, weak randomness) is an out-of-taxonomy control.

All 300 runs passed without timeouts, every fuzzing campaign completed its 10,000 calls, and each target’s exploit transaction consumed identical gas under all 30 seeds. The local execution back end is therefore stable and reproducible on commodity hardware at under half a second per run. Because the harnesses are hand-written, RQ2 does not measure how often generated harnesses find bugs (Sect. 6).

Table 3. Ablation of GNNMHA_{v2} on the random split (five seeds, mean $\pm$ std). Δ : change in mean macro-F1 relative to the full model.

Variant	Macro-F1	Δ	Top-2 accuracy
Full model	0.7595 ± 0.0019	–	0.8086
without edge attributes	0.7603 ± 0.0011	+0.0008	0.8085
without jumping knowledge	0.7597 ± 0.0018	+0.0002	0.8117
focal loss $\rightarrow$ cross-entropy	0.7584 ± 0.0012	–0.0010	0.8097
attention+mean $\rightarrow$ mean pooling	0.7596 ± 0.0029	+0.0001	0.8115
$L = 2$	0.7611 ± 0.0016	+0.0016	0.8079
$L = 3$	0.7588 ± 0.0025	–0.0007	0.8069
$K = 4$	0.7588 ± 0.0017	–0.0007	0.8089

Table 4. Foundry execution of reference harnesses over 30 seeds and two modes per target (60 runs each). Gas is per exploit transaction and the mean per fuzz call. Mean wall-clock time per run: 419.42 ms (directed exploit) and 479.59 ms (fuzzing).

Target	Class (SWC)	Exploit gas	Fuzz gas	Completed
SimpleDAO	Reentrancy (107)	77,558	49,602	60/60
TokenSale	Overflow (101)	39,648	35,249	60/60
UnprotectedVault	Access control (105)	48,550	42,252	60/60
UncheckedBank	Unchecked return (104)	27,202	50,029	60/60
WeakLottery	Weak randomness (120)	51,844	43,802	60/60
Total				300/300

5.3 RQ3: Pseudonymization Fidelity and Residual Exposure

Utility. Table 5 shows that DataMasker is lossless for compilation and restoration: every masked contract compiles with `solc`, and $\mathcal{T}^{-1}$ restores every contract exactly. For static analysis, we compare the set of Slither detector types raised on each contract before and after masking. All 229 contract-level detector types raised on the plaintext are raised on the masked code as well; masking adds six (235 in total), because renamed identifiers trigger naming-convention detectors.

Exposure. We quantify two A2 adversaries. The *dictionary* measure counts identifiers whose plaintext name is one of 32 common Solidity and DeFi terms (e.g., `owner`, `balance`, `withdraw`); an adversary who guesses these terms from the role of a pseudonym recovers at most 30.4% of identifiers. The *frequency* adversary knows how often each plaintext identifier occurs in the contract, for example from an earlier version or a leaked excerpt, ranks identifiers and pseudonyms by occurrence count, and aligns the two rankings. Because $\mathcal{T}$ renames one-to-one and preserves occurrence counts, only ties stop it, and it re-links 89.5% of identifiers. This is an upper bound under strong side knowledge, and it is the informative result: renaming preserves the structure of the code, so pseudonymization alone is not a confidentiality mechanism for code sent to an untrusted model, in line with learned name recovery from binaries [30, 31]. It is defense in depth for artifacts

Table 5. DataMasker on the 57 benchmark contracts (286 user-defined identifiers). Upper block: utility (higher is better). Lower block: exposure to an A2 adversary (lower is better).

Metric	Value	Meaning
Compilation validity	57/57 (100%)	masked code compiles
Exact restoration	57/57 (100%)	$\mathcal{T}^{-1}$ restores the source
Slither detector types retained	229/229 (100%)	per contract; 6 added
Dictionary exposure	87/286 (30.4%)	name in a 32-term domain list
Frequency re-linkage	256/286 (89.5%)	rank matching of counts

Table 6. Alarms along the static filtering stages on 57 contracts. “Safe flagged” and “Vuln. flagged” count contracts that retain at least one alarm.

Stage	Filter	Alarms	Safe flagged	Vuln. flagged
0	Slither, all detectors	230	27/27 (100%)	30/30 (100%)
1	Five target SWC classes only	41	6/27 (22.2%)	28/30 (93.3%)
2	High/medium impact, confidence	33	6/27 (22.2%)	27/30 (90.0%)

that leave the host, and this measurement is why PRISM keeps inference local instead of masking code and sending it to a hosted LLM.

5.4 RQ4: Alarm Reduction on Safe Contracts

We ran Slither 0.11.5 on the 57-contract benchmark (27 safe and 30 vulnerable contracts, labeled by their benchmark templates) and applied PRISM’s static filters in sequence (Table 6).

Unfiltered Slither raises 230 alarms and flags every contract, safe or not, so contract-level precision is 30/57 (52.6%); most alarms come from informational and style detectors. Restricting output to the five target classes removes 189 of the 230 alarms and leaves 6 of 27 safe contracts flagged, at the cost of two vulnerable contracts. Keeping only high- and medium-impact findings with high or medium confidence removes eight further alarms and one more vulnerable contract, but no safe contract. The remaining six safe contracts are the input for which execution-based confirmation is designed.

5.5 RQ5: Patch Validation

For the five case-study contracts with exploit harnesses (Table 4), template-based repair produced one patch each. Each patch was checked in five steps: (1) it compiles with `solc` 0.8.30; (2) it is minimal, changing at most 25% of the lines according to an abstract syntax tree (AST) parse; (3) the original exploit harness now reverts; (4) the 10,000-call fuzzing campaign still passes; and (5) function selectors and storage layout are unchanged. All five patches passed

all five checks, so each blocks its exploit while preserving the external application binary interface (ABI) and the storage layout, which matters for upgradeable proxies.

6 Discussion and Threats to Validity

Cost. Graph screening is linear in graph size and costs 0.21 ms per function after CPG extraction, so it is negligible next to one Foundry run of 0.42 to 0.48 s (Table 4). The LLM stage is invoked only for functions with $\Phi \geq \tau$, which bounds the number of generations per contract.

Threats to validity. *Construct.* Function graphs inherit contract-derived labels, so functions that do not contain the labeled flaw are counted as positives; RQ1 indicates that this, rather than the encoder, bounds accuracy. The 57 benchmark labels come from the benchmark templates rather than from independent source-span validation, and the SWC-114 contracts in that benchmark are timestamp-dependence proxies rather than validated front-running cases. Real exploits may also compound several weaknesses. *Internal.* The end-to-end path from ranking through LLM audit to execution-confirmed findings has not yet been measured on the 57 contracts: RQ2 uses hand-written harnesses, the current invariant templates are specific for reentrancy and access control but not yet for integer overflow and unchecked return, and LLM findings are reported as unconfirmed hypotheses. Graph extraction depends on Slither IR; inline Yul and compilers outside 0.4.x to 0.8.x may degrade CPG quality. DataMasker leaves comments and string literals unchanged, so they can reveal business logic in shared artifacts. *External.* Training on SolidiFI and testing on LISA-Bench without adaptation lowered macro-F1 from 0.8985 to 0.5157, so transfer across coding conventions remains open. The SWC registry is no longer actively maintained, and the RQ2 to RQ5 contracts are small reference contracts compiled with `solc` 0.8.30, where arithmetic overflow requires `unchecked` blocks.

Data availability. Code, configuration, frozen split manifests, result files and reference harnesses are available in an anonymized repository: <https://anonymous.4open.science/r/PRISM-PROJECT/>.

7 Conclusion

This paper presented PRISM, a confidential smart contract audit pipeline that keeps every model and artifact on the audit host. Its two-stage graph attention ranker over typed SSA-level function graphs orders functions at $76.31 \pm 0.06\%$ macro-F1 on unseen contract lineages in 0.21 ms per function, and the finding that graph learners of different design share almost all of their errors locates the remaining gap in contract-derived labels rather than in model capacity. Its reversible pseudonymization is lossless for compilation, restoration and static

analysis, and the measured 89.5% frequency re-linkage turns the choice of local inference from a preference into a requirement. Together with a reproducible local EVM back end and filtering that reduces flagged safe contracts from 27/27 to 6/27, these results make confidential, execution-backed auditing a practical workflow for teams that cannot send code to a model provider.

Future work will measure the complete pipeline end to end on larger real-world benchmarks, with LLM findings compiled into class-specific invariants and confirmed or refuted by execution; will replace contract-derived supervision with function-level labels from audit reports and execution traces to lift the accuracy ceiling identified here; and will replace template repair with LLM-guided patch synthesis validated by the same five checks.

References

1. Atzei, N., Bartoletti, M., Cimoli, T.: A survey of attacks on Ethereum smart contracts (SoK). In: International Conference on Principles of Security and Trust (POST). LNCS, vol. 10204, pp. 164–186. Springer (2017)
2. Brody, S., Alon, U., Yahav, E.: How attentive are graph attention networks? In: International Conference on Learning Representations (ICLR) (2022)
3. Choi, J., Kim, D., Kim, S., Grieco, G., Groce, A., Cha, S.K.: Smartian: Enhancing smart contract fuzzing with static and dynamic data-flow analyses. In: 36th IEEE/ACM International Conference on Automated Software Engineering (ASE). pp. 227–239 (2021)
4. Durieux, T., Ferreira, J.F., Abreu, R., Cruz, P.: Empirical review of automated analysis tools on 47,587 Ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE). pp. 530–541 (2020)
5. Feist, J., Grieco, G., Groce, A.: Slither: A static analysis framework for smart contracts. In: IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB). pp. 8–15 (2019)
6. Ferreira Torres, C., Jonker, H., State, R.: Elysium: Context-aware bytecode-level patching to automatically heal vulnerable smart contracts. In: Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses (RAID) (2022)
7. Gai, K., Liang, H., Yu, J., Zhu, L., Niyato, D.: LLAMA: Multi-feedback smart contract fuzzing framework with LLM-guided seed generation. *IEEE Transactions on Information Forensics and Security* (2026)
8. Ghaleb, A., Pattabiraman, K.: How effective are smart contract analysis tools? evaluating tools on solidifi benchmark. *arXiv preprint arXiv:2009.00000* (2020)
9. Grieco, G., Song, W., Cygan, A., Feist, J., Groce, A.: Echidna: Effective, usable, and fast fuzzing for smart contracts. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). pp. 557–560 (2020)
10. Jiang, B., Liu, Y., Chan, W.K.: Contractfuzzer: Fuzzing smart contracts for vulnerability detection. In: Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering (ASE). pp. 259–269 (2018)
11. Li, C., Gim, I., Zhong, L.: Confidential prompting: Privacy-preserving LLM inference on cloud. *arXiv preprint arXiv:2409.19134* (2024)

12. Lin, X., Xie, Q., Zhao, B., Tian, Y., Zonouz, S., Ruan, N., Li, J., Beyah, R., Ji, S.: PromFuzz: Leveraging LLM-driven and bug-oriented composite analysis for detecting functional bugs in smart contracts. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE). pp. 1044–1056 (2025)
13. LISA-Bench Team: LISA-Bench: A benchmark for smart contract vulnerability auditing. <https://github.com/anonymous/lisa-bench> (2024)
14. Liu, Y., Xue, Y., Wu, D., Sun, Y., Li, Y., Shi, M., Liu, Y.: PropertyGPT: LLM-driven formal verification of smart contracts through retrieval-augmented property generation. In: Network and Distributed System Security Symposium (NDSS) (2025)
15. Liu, Z., et al.: Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Transactions on Knowledge and Data Engineering* (2021)
16. Luu, L., Chu, D.H., Olickel, H., Saxena, P., Hobor, A.: Making smart contracts smarter. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 254–269 (2016)
17. Ma, Z., et al.: iAudit: Automated smart contract auditing with llm-based agents. arXiv preprint arXiv:2501.00000 (2025)
18. Mandana, E., Vlahavas, G., Vakali, A.: EVuLLM: Ethereum smart contract vulnerability detection using large language models. *Electronics* **14**(16), 3226 (2025)
19. Nguyen, T., et al.: MANDO-LLM: Heterogeneous graph transformers with llm-derived features for smart contract auditing. *IEEE Transactions on Software Engineering* (2026)
20. Nguyen, T.D., Pham, L.H., Sun, J.: sGuard: Towards fixing vulnerable smart contracts automatically. In: IEEE Symposium on Security and Privacy (S&P) (2021)
21. Nguyen, T.D., Pham, L.H., Sun, J., Lin, Y., Minh, Q.T.: sfuzz: An efficient adaptive fuzzer for Solidity smart contracts. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE). pp. 778–788 (2020)
22. Pang, Y., Liu, X., Huang, T., Hong, Y., Huang, J., Duan, S., Dong, C.: Graph-based contract sensing framework for smart contract vulnerability detection. *IEEE Transactions on Big Data* (2025). <https://doi.org/10.1109/TBDATA.2025.3594303>
23. Shou, C., Tan, S., Sen, K.: ItyFuzz: Snapshot-based fuzzer for smart contract. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA) (2023). <https://doi.org/10.1145/3597926.3598059>
24. Sun, Y., Wu, D., Xue, Y., Liu, H., Wang, H., Xu, Z., Xie, X., Liu, Y.: GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE) (2024). <https://doi.org/10.1145/3597503.3639117>
25. Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., Vechev, M.: Securify: Practical security analysis of smart contracts. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 67–82 (2018)
26. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. *International Conference on Learning Representations (ICLR)* (2018)
27. Wang, S.J., Pei, K., Yang, J.: SmartInv: Multimodal learning for smart contract invariant inference. In: IEEE Symposium on Security and Privacy (S&P) (2024)
28. Wei, Z., Sun, J., Zhang, Z., Zhang, X., Li, M., Hou, Z.: LLM-SmartAudit: Advanced smart contract vulnerability detection. arXiv preprint arXiv:2410.09381 (2024)

29. Wüstholtz, V., Christakis, M.: Harvey: A greybox fuzzer for smart contracts. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). pp. 1398–1409 (2020)
30. Xie, Z., et al.: ReSym: Recovering variable names and types from stripped binaries with large language models. In: Proceedings of the ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS) (2024)
31. Xu, H., et al.: GenNm: Generative identifier name recovery via deep learning. In: IEEE/ACM International Conference on Software Engineering (ICSE) (2025)
32. Zhen, Z., et al.: DA-GNN: A smart contract vulnerability detection method based on dual attention graph neural network. *Computer Networks* **242**, 110238 (2024)
33. Zhu, Q., et al.: DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931* (2024)
34. Zhuang, Y., Liu, Z., Qian, P., Liu, Q., Wang, X., He, Q.: Smart contract vulnerability detection using graph neural networks. In: Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI). pp. 3283–3290 (2021)